Owen Manley

Professor Ghazaleh

CSC 450

May 6th, 2025

Assignment 8

1- List 3 examples of deadlocks that are not related to a computer-system environment.

- Four cars approach a four-way intersection. Each driver was not paying attention to who arrived first so they all stop waiting for someone to go first. Causes deadlock because each car are waiting on one another to proceed.
- Three coworkers (Bob, Alex, and Rob), work office jobs. Bob realizes he needs Rob's paperwork to finish his own, Alex needs Bob's paperwork to finish, and Rob needs Alex's paperwork. Each coworker can't finish their paperwork because they require resources that the other co workers are working on. Thus, causing deadlock because they all can't finish their paperwork since they need different resources to complete that are not finished themselves.
- Two children want to play with each others toys but both are thinking of not giving their toys up, causing deadlock because both kids are not willing to give up their toys to play with the one they want.

2- How can the hold-and-wait condition be prevented?

- One way to prevent the hold-and-wait condition is by eliminating the hold condition. For instance, a process must release the resources that it is holding in order to request another one from another process.
- Eliminating the wait condition could also work by each process specifically specifying the resources it needs so that it does not need to wait for allocation.

3- Is it possible to have a deadlock involving only one process? Explain your answer.

Deadlock is not possible with only one process. Furthermore, it is only possible if there are multiple processes requiring the same resources.

4- Consider a system consisting of 4 resources of the same type (a printer for example) that are shared by 3 processes, each of which needs at most 2 resources. Show that the system is deadlock-free.

Resources (r) = 4

Processes (p)= 3

Need = 2R

The printer is deadlock-free through the following. P1, P2, and P3 will all hold 1 resource, with 1 being left out not being used. A process will then take the resource so that it satisfies its need of two resources in order to complete its task. Once the task is complete, the resources are released, leaving two resources to be used. Since 1 process is already satisfied, 2 are left. One resource will be sent to both the processes, allowing them to complete their tasks deadlock-free.